40009 312

Kotlin Search Engine

Submitters

---

kw1425

10·5
16

Emarking

```
Final Tests: Summary for kw1425 of c1
PPT 21
-------------------------------------

  Public Tests:
    Compiles:     1 / 1
    Tests Pass:   1 / 1
    Style Check:  1 / 1

Git Repo: git@gitlab.doc.ic.ac.uk:lab2526_autumn/kotlinsearchengine_kw1425.git
Commit ID: f9da9
Git Diff: https://gitlab.doc.ic.ac.uk/lab2526_autumn/kotlinsearchengine_kw1425/-/compare/master..f9da9?from_project_id=134877
```

```
 1: package websearch
 2:
 3: import org.jsoup.Jsoup.parse
 4: import java.io.File
 5:
 6: val downloadedWebPages: Map<URL, WebPage> =
 7:   mapOf(
 8:     URL("https://kotlinlang.org/") to loadPage("kotlinlang.html"),
 9:     URL("https://research.google/pubs/pub62/") to loadPage("mapreduce.html"),
10:     URL("https://wiki.haskell.org/Introduction") to loadPage("haskellwiki.html")
11:   )
12:
13: private fun loadPage(filename: String) = WebPage(parse(File(
"src/main/data/$filename").bufferedReader().readLines().joinToString("\n")))
```

```
 1: package websearch
 2:
 3: import org.jsoup.Jsoup.connect
 4: import org.jsoup.Jsoup.parse
 5: import org.jsoup.nodes.Document
 6: import org.jsoup.select.Elements
 7:
 8: fun main() {
 9:
10:   val htmlString =
11:     """<html>
12:          <head>
13:            <title>A Simple HTML Document</title>
14:          </head>
15:            <body>
16:              <p>This is a very simple <a href=↙
"https://en.wikipedia.org/wiki/HTML">HTML</a> document.</p>
17:              <p>It only has   two    paragraphs.</p>
18:            </body>
19:        </html>"""
20:
21:   // parse a string into a Document using Jsoup.parse()
22:   val htmlDocument: Document = parse(htmlString)
23:   println(WebPage(htmlDocument).extractWords())
24:
25:   // get all the text from the document
26:   println(htmlDocument.text())
27:
28:   // extract all the <a >...</a> tags (which create links)
29:   val aTags: Elements = htmlDocument.getElementsByTag("a")
30:   println(aTags)
31:
32:   // get the value of the href attribute (where the link points to)
33:   val firstTag = aTags.first()
34:
35:   // aTags.first() could return null, so we have to check that before using it.
36:   if (firstTag != null) {
37:     val link: String = firstTag.attr("href")
38:     println(link)
39:   }
40:
41:   // A different approach to dealing with the nullable value is to use the ↙
"safe-call" operator ?.
42:   // This says, if firstTag is null, don't call .attr(), just return null.
43:   println(firstTag?.attr("href"))
44:
45:   // You can also use the "Elvis" operator ?: to use a specific value if the ↙
safe-call returns null
46:   println(firstTag?.attr("href") ?: "")
47:
48:   // do the same thing using a CSS selector
49:   val selection: Elements = htmlDocument.select("a[href]")
50:   val firstResult = selection.first()
51:
52:   println(firstResult?.attr("href"))
53:   println(firstResult?.attr("href") ?: "")
54:
55:   // download a document from the internet using Jsoup.connect()
56:   val downloadedDocument = connect("https://kotlinlang.org/").get()
57:   println(downloadedDocument.body())
58: }
```

```
 1: package websearch
 2:
 3: class SearchEngine(
 4:   val corpus: Map<URL, WebPage>
 5: ) {
 6:   private var index: Map<String, List<SearchResult>> = mutableMapOf()
 7:
 8:   fun rank(urls: List<URL>): List<SearchResult> {
 9:     val frequency = urls.groupBy { it.url }.mapValues { it.value.size }
10:     return frequency.map { entry ->
11:       val url = URL(entry.key)
12:       val freq = entry.value
13:       SearchResult(url, freq)
14:     }
15:   }
16:
17:   fun compileIndex() {
18:     val wordsOnPage: MutableList<Pair<String, URL>> = mutableListOf()
19:     corpus.forEach { (url, page) ->
20:       page.extractWords().forEach { word ->
21:         wordsOnPage.add(word to url)
22:       }
23:     }
24:
25:     index =
26:       wordsOnPage
27:         .groupBy(
28:           { it.first },
29:           { it.second }
30:         ).mapValues { entry -> rank(entry.value).sortedByDescending { it.numRefs
} }
31:   }
32:
33:   fun searchFor(query: String): SearchResultsSummary {
34:     val results = index[query] ?: emptyList()
35:     return SearchResultsSummary(query, results)
36:   }
37: }
38:
39: class SearchResult(
40:   val url: URL,
41:   val numRefs: Int
42: )
43:
44: class SearchResultsSummary(
45:   val query: String,
46:   val results: List<SearchResult>
47: ) {
48:   override fun toString(): String =
49:     buildString {
50:       appendLine("Results for $query:")
51:       results.forEach { result ->
52:         appendLine("${result.url}- ${result.numRefs} references")
53:       }
54:     }
55: }
```

1.5/2

→ look into groupingBy and eachCount()

→ equivalent to: frequency.map { (url, freq) -> SearchResult(URL(url), freq) }

} use flatMap to shorten this:

val WOP = corpus.flatMap { (url, page) ->
page.extractWords().map { it -> Pair(it, url) } }

2/2

can be called in rank function instead.

2/2

✓

3/3

✓

```
 1: package websearch
 2:
 3: import org.jsoup.nodes.Document
 4:
 5: class URL(
 6:   val url: String
 7: ) {
 8:   override fun toString(): String = url
 9: }
10:
11: class WebPage(
12:   val content: Document
13: ) {
14:   fun extractWords(): List<String> {
15:     val words =
16:       content
17:         .text()
18:         .lowercase()
19:         .split(" ", ".", ",")
20:         .filter { it.isNotBlank() }
21:     return words
22:   }
23: }
```

✓

2/2

```
package websearch

import org.jsoup.Jsoup
import org.junit.Test
import kotlin.test.assertEquals

class SearchEngineTest {
  private val docHomePageHtml =
    """<html>
         <head>
           <title>Department of Computing</title>
         </head>
           <body>
             <p>Welcome to the Department of Computing at <a href="https://www.imperial.ac.uk">Imperial College London</a>.</p>
           </body>
       </html>"""

  private val imperialHomePageHtml =
    """<html>
         <head>
           <title>Imperial College London</title>
         </head>
           <body>
             <p>Imperial people share ideas, expertise and technology to find answers to the big scientific questions and tackle global challenges</p>
             <p>See the latest news about our research on the <a href="https://www.bbc.co.uk/news">BBC</a></p>
           </body>
       </html>"""

  private val bbcNewsPageHtml =
    """<html>
         <head>
           <title>BBC News</title>
         </head>
           <body>
             <p>Here is all the latest news about science and other things.</p>
           </body>
       </html>"""

  private val docHomePage = WebPage(Jsoup.parse(docHomePageHtml))
  private val imperialHomePage = WebPage(Jsoup.parse(imperialHomePageHtml))
  private val bbcNews = WebPage(Jsoup.parse(bbcNewsPageHtml))

  private val downloadedPages =
    mapOf(
      URL("https://www.imperial.ac.uk/computing") to docHomePage,
      URL("https://www.imperial.ac.uk") to imperialHomePage,
      URL("https://www.bbc.co.uk/news") to bbcNews
    )

  @Test
  fun `can index downloaded pages`() {
    val searchEngine = SearchEngine(downloadedPages)
    searchEngine.compileIndex()
    val summary = searchEngine.searchFor("news")

    assertEquals("news", summary.query)
    assertEquals(2, summary.results.size)

    assertResultsMatch("https://www.bbc.co.uk/news", 2, summary.results[0])
    assertResultsMatch("https://www.imperial.ac.uk", 1, summary.results[1])
  }

  private fun assertResultsMatch(
```



```
    expectedUrl: String,
    numRefs: Int,
    searchResult: SearchResult
  ) {
    assertEquals(expectedUrl, searchResult.url.toString())
    assertEquals(numRefs, searchResult.numRefs)
  }
}
```

```
 1: -------- Test Output --------
 2: Running LabTS build... (Fri 28 Nov 18:37:47 UTC 2025)
 3:
 4: Submission summary...
 5: You made 1 commits
 6:   - f9da919 Finish [5 files changed, 91 insertions, 9 deletions]
 7:
 8: Preparing...
 9:
10: BUILD SUCCESSFUL in 893ms
11:
12: Compiling...
13: BUILD SUCCESSFUL in 10s
14:
15: Running tests...
16:
17: SearchEngineTest > can index downloaded pages PASSED
18:
19: WebPageTest > extracts words from page PASSED
20:
21: WebPageTest > converts case and removes punctuation PASSED
22:
23: BUILD SUCCESSFUL in 1s
24:
25: Checking code style...
26: BUILD SUCCESSFUL in 2s
27: Finished auto test. (Fri 28 Nov 18:38:29 UTC 2025)
28:
29: -------- Test Errors --------
30:
```